252-0027

Einführung in die Programmierung
Übungen

Woche 5: Arrays, Methoden, Debugger

Fabian Rohlik
Departement Informatik
ETH Zürich

Feedback zu letzter Abgabe

- Von vielen sehr gut gelöst
- macht EProg-Programmieraufgaben
  (selber coden ist nochmal anders als die Theorie zu verstehen)
- EProg wird schwieriger, bleibt dran

Bonus ab dieser Woche

- macht die Aufgabe so früh wie möglich, und macht öfters Commit & Push
- macht keine Annahmen, folgt genau der Aufgabenstellung
- schaut auf GitLab nach, ob die Abgabe funktioniert hat

Plan heute

- Wiederholung von letzter Woche
- Arrays
- Method Overloading
- Value vs Reference

If / Else Übung (Prüfungsaufgabe)

Für diese Aufgabe betrachten wir folgende Methode:

```
public int f(int x, int y) {
  // body
}
```

(a)
```
if (x < y) {
  return 1;
}
return 0;
```

(b)
```
if (y > x) {
  return 0;
} else {
  return 1;
}
```

„Geben Sie an, ob (a) und (b) für Werte x und y immer, nie, oder manchmal das gleiche zurückgeben.“

If / Else Übung (Prüfungsaufgabe)

Für diese Aufgabe betrachten wir folgende Methode:

```
public int f(int x, int y) {
  // body
}
```

(a)
```
if (x < y) {
  return 1;
}
return 0;
```

(b)
```
if (y > x) {
  return 0;
} else {
  return 1;
}
```

N

If / Else Übung (Prüfungsaufgabe)

Für diese Aufgabe betrachten wir folgende Methode:

```
public int f(int x, int y) {
  // body
}
```

(a)
```
if (x < y) {
  return 1;
}
return 0;
```

(d)
```
if (x > y) {
  return 0;
}
return 1;
```

If / Else Übung (Prüfungsaufgabe)

Für diese Aufgabe betrachten wir folgende Methode:

```
public int f(int x, int y) {
  // body
}
```

(a)
```
if (x < y) {
    return 1;
}
return 0;
```

(d)
```
if (x > y) {
    return 0;
}
return 1;
```

M
(der Fall == funktioniert nicht)

If / Else Übung (Prüfungsaufgabe)

Für diese Aufgabe betrachten wir folgende Methode:

```
public int f(int x, int y) {
  // body
}
```

(d)
```
if (x > y) {
  return 0;
}
return 1;
```

(e)
```
if (x > y) {
  return 0;
} else {
  return 1;
}
```

If / Else Übung (Prüfungsaufgabe)

Für diese Aufgabe betrachten wir folgende Methode:

```
public int f(int x, int y) {
  // body
}
```

(d)
```
if (x > y) {
    return 0;
}
return 1;
```

(e)
```
if (x > y) {
    return 0;
} else {
    return 1;
}
```

Nach dem return hört die Ausführung einer Methode auf.

1D Arrays

```java
public static void main() {

    int zahl1 = 5;
    int zahl2 = 10;
    int zahl3 = 15;
    int zahl4 = 20;
    // ...

}
```

```java
public static void main() {

    int zahl1 = 5;
    int zahl2 = 10;
    int zahl3 = 15;
    int zahl4 = 20;
    // ...

}
```

```java
public static void main() {
    int[] zahlen = {5, 10, 15, 20};
}
```

Eindimensionale Arrays

- Arrays haben eine **feste Länge**
- Jeder Eintrag hat speichert einen Wert des deklarierten Datentyps
- auf ein spezifisches Element **i** zugreifen können wir mit **arr[i]**
- Indizierung startet bei **0**, geht also bis **n-1**

```java
public class Main {
    public static void main(String[] args){
        int[] arr = {3,5,6,1,2,8};
        for(int i = 0; i < arr.length; i++){
            arr[i] *= 2;
        }
    }
}
```

Eindimensionale Arrays

- Arrays haben eine **feste Länge**
- Jeder Eintrag hat speichert einen Wert des deklarierten Datentyps
- auf ein spezifisches Element **i** zugreifen können wir mit **arr[i]**
- Indizierung startet bei **0**, geht also bis **n-1**

```
int[] arr = {3,5,6,1,2,8};
```

```
int[] arr = new int[10];
```

```
arr[4] = 10;
```

Index vs Länge

Wie bei Strings:

- Länge ist die „intuitive“ Länge, beim „Zählen“ beginnen wir bei 1
- Index beginnt bei 0

```
public static void main() {

    // array numbers hat length 4

    // index:       0   1   2   3
    int[] numbers = {5, 10, 15, 20};
}
```

2D Arrays

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

[[1,2,3], [4,5,6], [7,8,9]]

[[1], [4,5,6,6,7], [7,8]]

[[1], null, [7,8]]

Zweidimensionale Arrays

- eine Matrix könnte als Zweidimensionales Array dargestellt werden
- **int[][] arr = new int[n][n];**

  definiert ein Array, welches **n** integer-Arrays der Länge **n** speichert **(nxn-Matrix)**
- eine gesamte Zeile erhalten wir also mit **arr[i]** für **0≤i<n**
- einen Eintrag erhalten wir mit **arr[i][j]** für **0≤i,j<n**
- **Vorsicht:** Die Längen der Arrays könnten unterschiedlich sein!

```java
public class Main {
    public static void main(String[] args) {
        int[][] arr = {{1, 2, 3},{4, 5, 6},{7, 8, 9}};
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++) {
                arr[i][j] *= 2;
            }
        }
    }
}
```

2D Arrays

array = {arrOne, arrTwo, arrThree}

i	array[i]
0	
1	
2	

i	0	1	2	3	4	5
arrOne[i]	4	501	-1	200	42	0

i	0	1	2
arrTwo[i]	11	21	-170

i	0	1	2	3
arrThree[i]	100	1	-321	3

```
int[][] arTwoD = {
            {1,2,3},
            {4,5,6},
            {7,8,9}
};
```

arTwoD[0] = {1, 2, 3}

arTwoD[0][2] = 3

Macht keine Annahmen über die Längen

- IntelliJ Demo

Weitergabe von Referenzen

- ***numbers*** speichert die Referenz, wo die tatsächlichen Werte liegen
- gib Referenz an ***modifyArray*** weiter
- ***modifyArray*** sucht Werte im Speicher, verdoppelt sie, terminiert
- ***main*** sucht mit derselben im Speicher und findet verdoppelte Werte **ohne**, dass wir sie zurückgeben mussten

-> Siehe IntelliJ Demo

```java
public class ReferenceArrayDemo {
    public static void main(String[] args) {
        // Erstelle ein Array
        int[] numbers = {1, 2, 3, 4, 5};
        System.out.println("Vor der Änderung:");
        System.out.println(Arrays.toString(numbers));
        modifyArray(numbers); // Übergabe des Arrays
        System.out.println("Nach der Änderung:");
        System.out.println(Arrays.toString(numbers));
    }
    public static void modifyArray(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            arr[i] *= 2; // Verdopple jeden Wert im Array
        }
    }
```

```
static void main(){
    int[] numbers = new int[5]
    increase(numbers)
}

public static void increase(int[] zahlen){
    zahlen[2]=10;
    zahlen = new int[]{8,8,8};
    zahlen[0]=2;
}
```



Primitive Datentypen

- Variablen speichern einen Wert, z. B. *int*, *double* oder *boolean*
- Haben einen fest definierten Wertebereich
- Standardwerte sind festgelegt
- Können nicht *null* sein

Referenztypen

- Variablen speichern eine Referenz auf ein Objekt (z.B. String oder Array) oder *null*
- Objekte können unterschiedlich viel Speicher benötigen
- Standardwert ist *null*

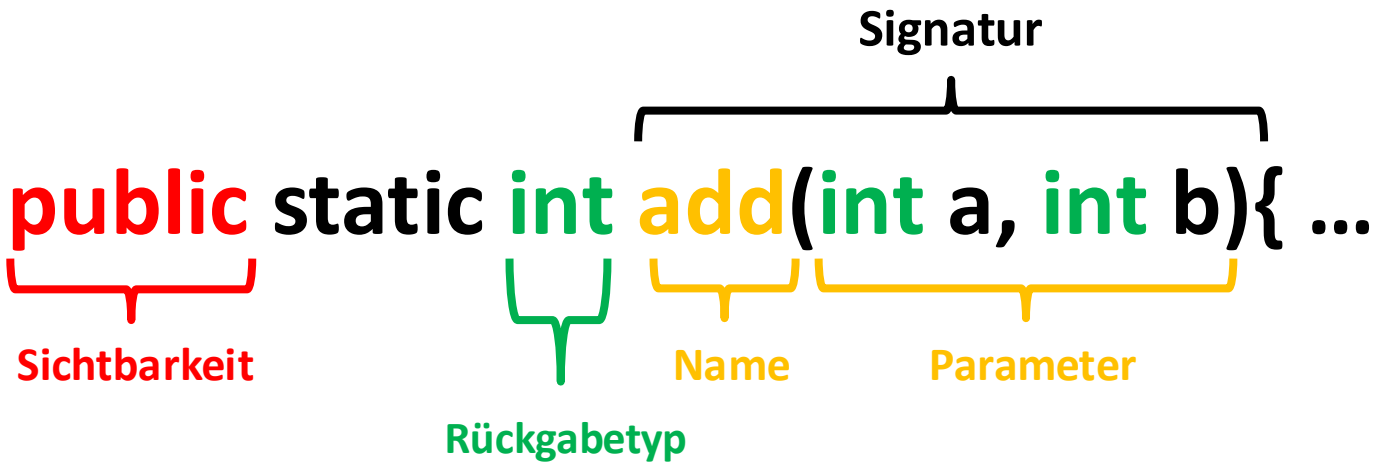
Aufgabe

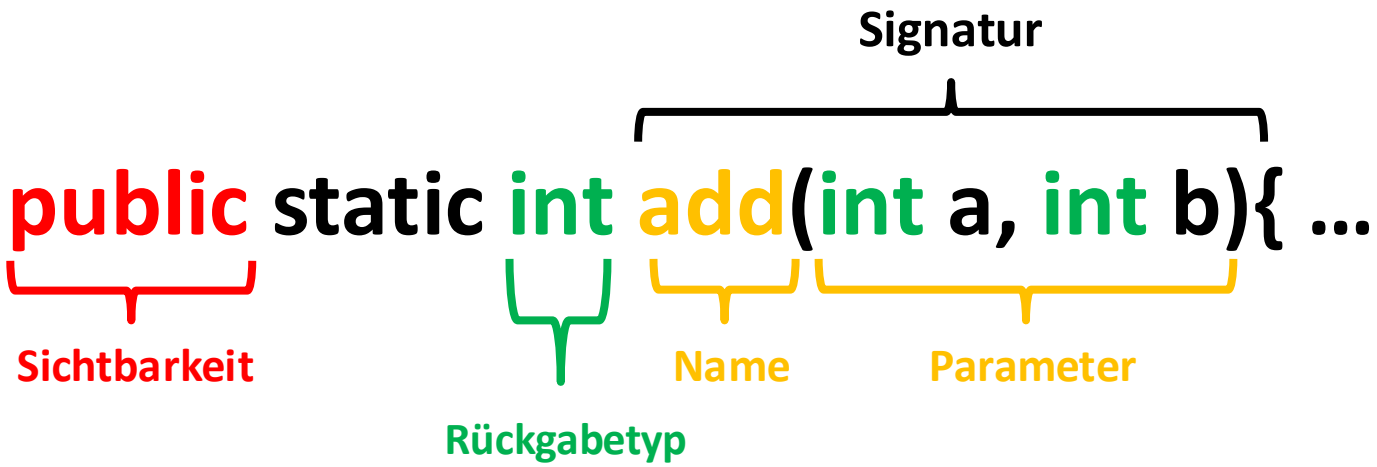
```
public class ArrayReference {
    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        int[][] b = { a, a, {4, 5, 6}};
        b[0][0] = 11;

        System.out.println(b[1][0]);
    }
}
```

Methoden

```
public static int add(int a, int b){ …
```


Signatur
public static int add(int a, int b){ …
Sichtbarkeit
Rückgabetyp
Name
Parameter



- Static: Methode gehört zur Klasse selbst und nicht zu einem bestimmten Objekt (einer Instanz) dieser Klasse
- Public: kann von überall im gesamten Programm aufgerufen werden

Method-Overloading

- Eine Methode kann überladen werden, in dem die Methodenparameter verändert werden
- trotz gleichem Namen hat sie dann eine andere Signatur
- **add** kann durch Überladung mit unterschiedlichen Parametern aufgerufen werden, hat nun unterschiedliche Rückgabetypen

```java
public class Main {
    // Erste Version: Addiert zwei int-Werte
    public static int add(int a, int b) {
        return a + b;
    }
    // Überladene Version: Addiert drei int-Werte
    public static int add(int a, int b, int c) {
        return a + b + c;
    }
    // Überladene Version: Addiert zwei double-Werte
    public static double add(double a, double b) {
        return a + b;
    }
}
```

Rekursion

Base-Case (“einfacher return”) nicht vergessen

vgl. EBNF

Rekursion - Beispiel

1. Implementieren Sie die Methode `Calculations.checksum(int x)`, das heisst die Methode `checksum` in der Klasse `Calculations`. Die Methode nimmt einen Integer `x` als Argument, welcher einen nicht-negativen Wert hat. Die Methode soll die Quersumme von `x` zurückgeben. Sie sollen für diese Aufgabe **keine** Schleife verwenden.

   Beispiele

   - `checksum(258)` gibt 15 zurück.
   - `checksum(49)` gibt 13 zurück.
   - `checksum(12)` gibt 3 zurück.

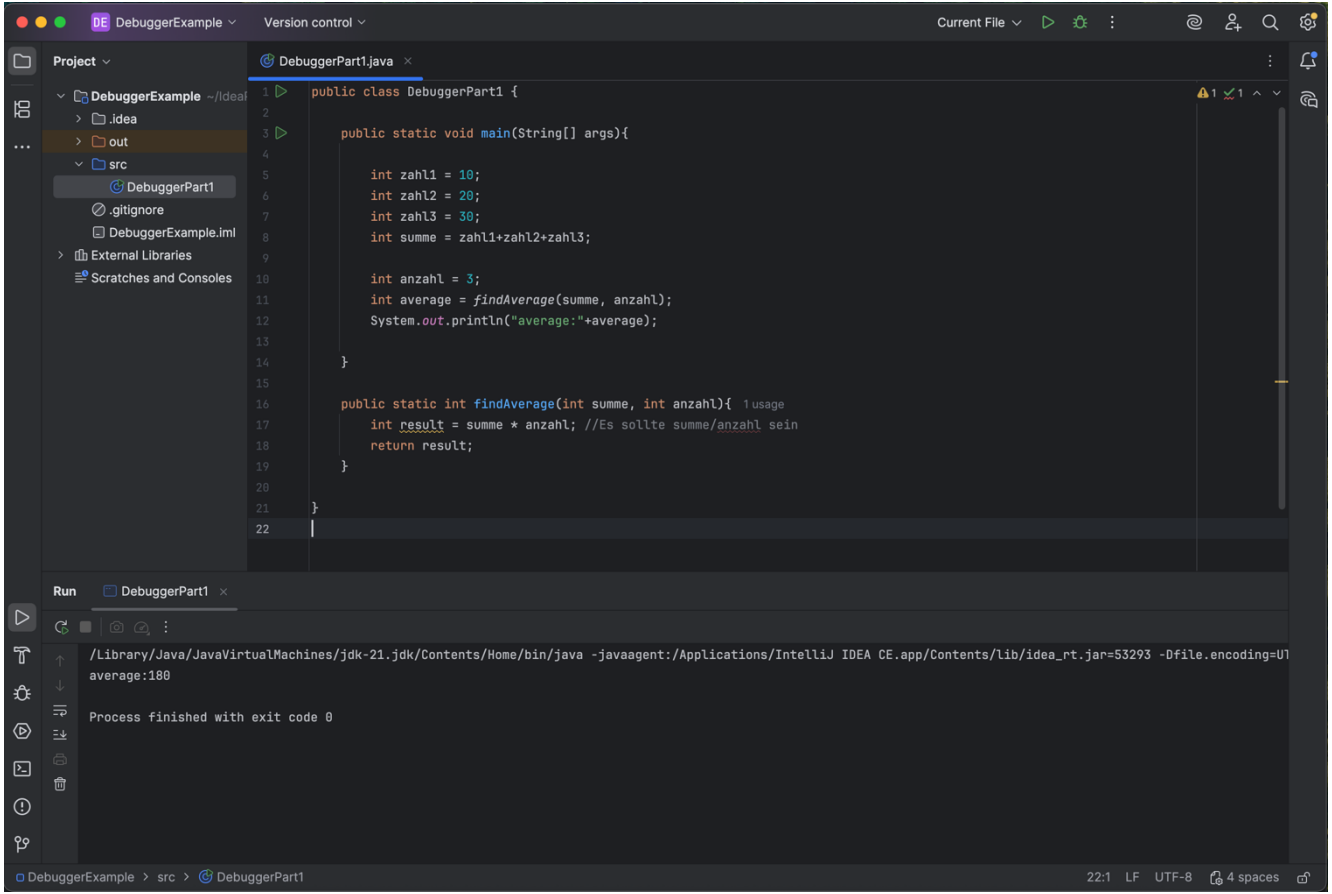
   Hinweis: Für einen Integer a ist `a % 10` die letzte Ziffer und `a / 10` entfernt die letzte Ziffer. Zum Beispiel `258 % 10` ist 8 und `258 / 10` ist 25.

cs.rohlik.net -> EProg Übungsstunde -> Kapitel 6

Java Debugger

Was ist der Debugger und was tut er?

- Ein Tool (in Java), das beim Debuggen hilft.
- Mit dem Java-Debugger kann man Schritt für Schritt durch das Programm gehen und genau beobachten, wie es ausgeführt wird.
- Die Änderungen der Variablen und ihrer Werte in jeder Zeile werden in einer Tabelle dargestellt.



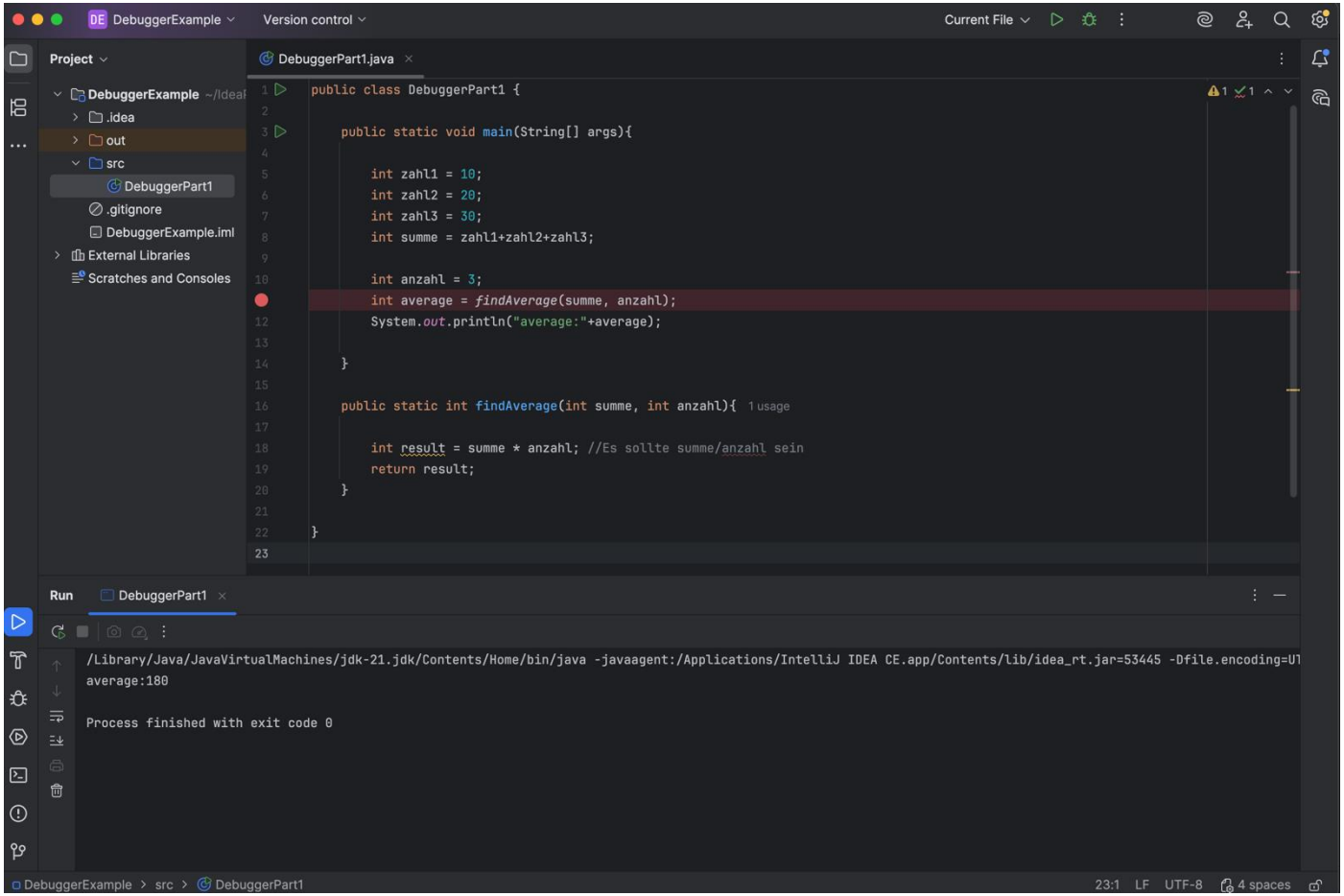
Das Programm macht nicht, was du erwartest, und du kannst den Fehler nicht finden?

Benutze den Debugger!

Breakpoints

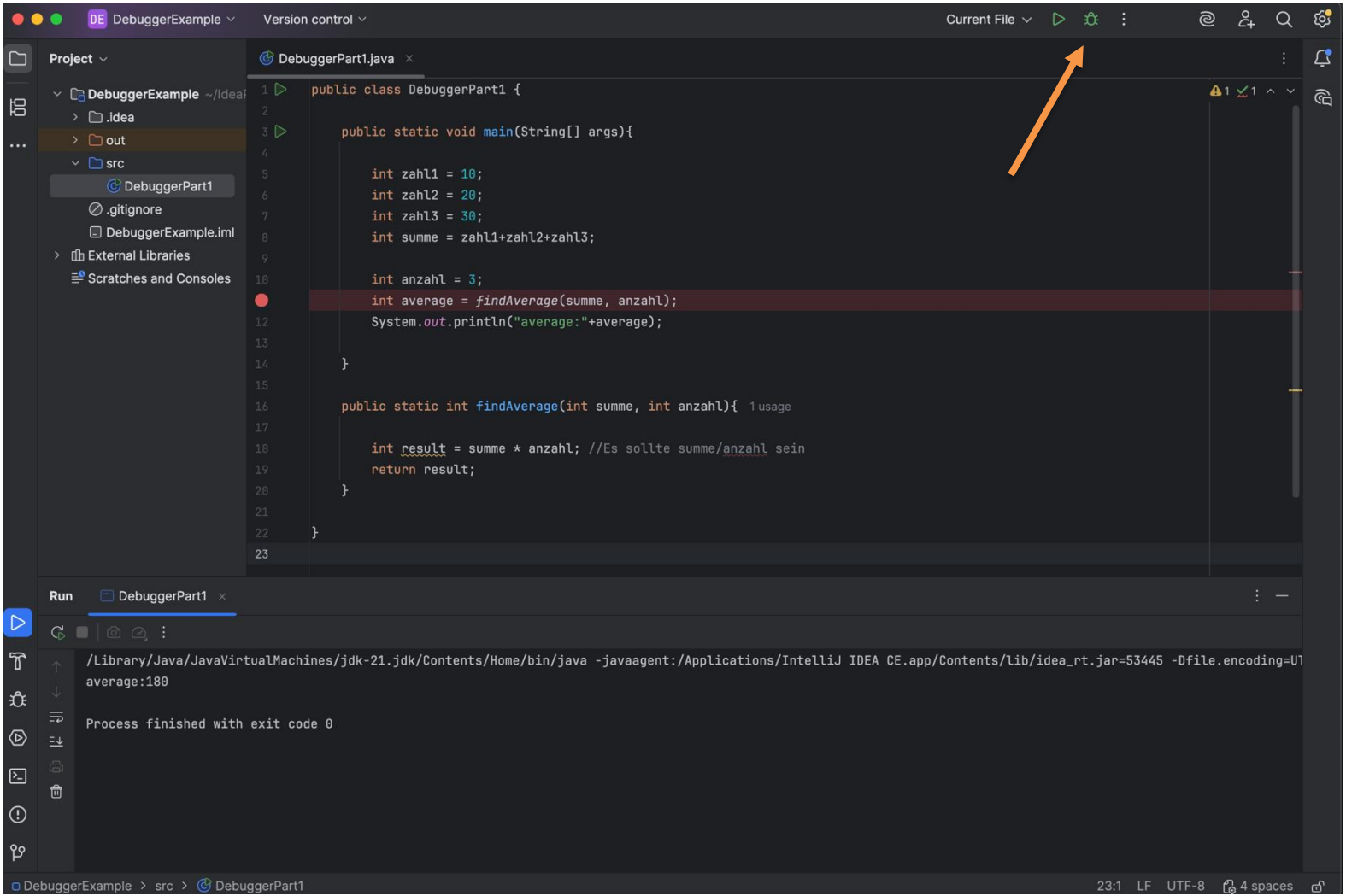
- Bis zum Breakpoint wird alles automatisch ausgeführt, und sobald die Zeile mit dem Breakpoint erreicht wird, stoppt die automatische Ausführung.
- Anschliessend kann der Benutzer Zeile für Zeile das Programm selbst ausführen.
- **Breakpoint auswählen:** An einer Stelle, an der man weiss, dass alles bis dahin wie erwartet funktioniert.
- **Breakpoint setzen:** Links von der Spalte mit den Zeilennummern klicken, um den Breakpoint zu setzen

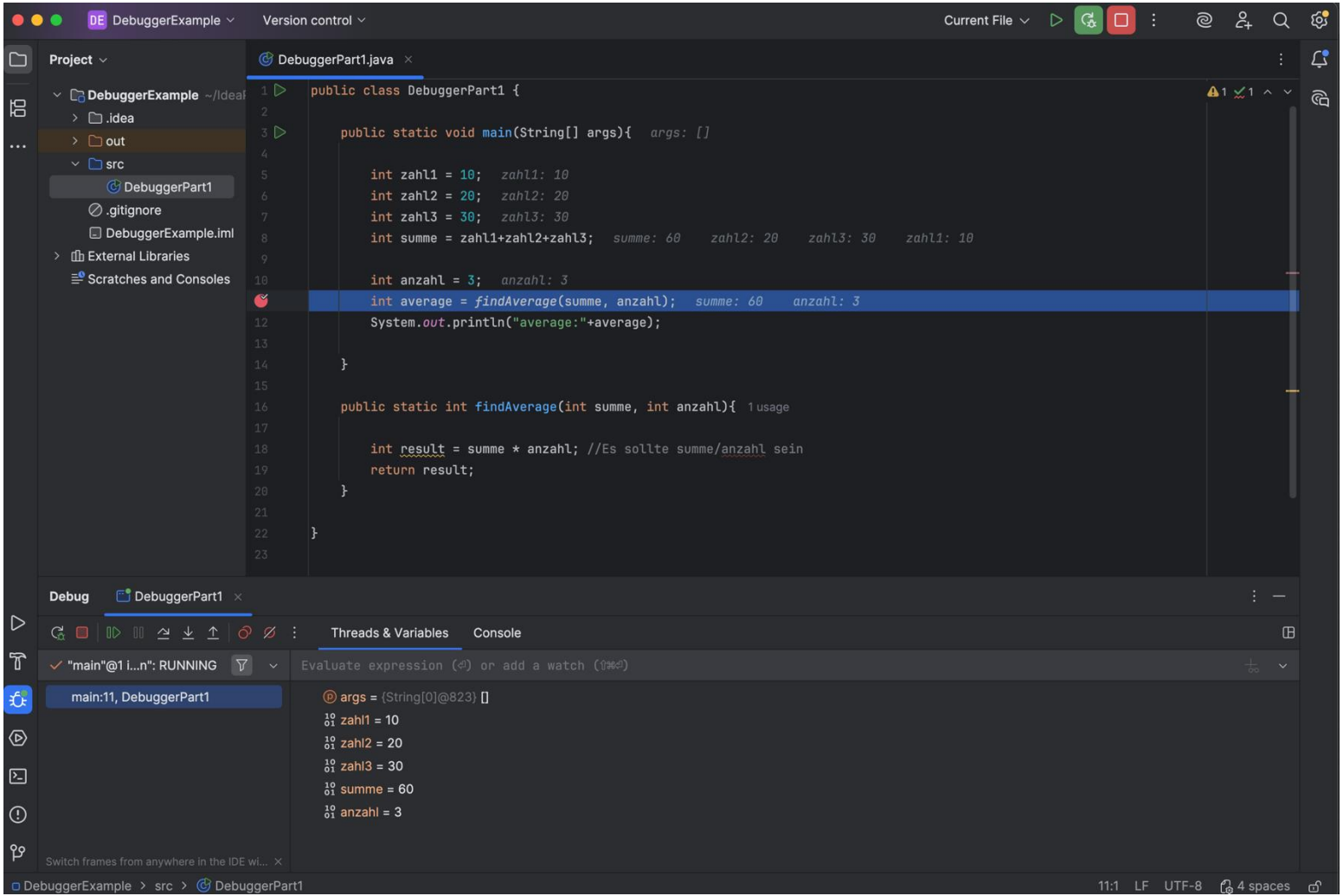




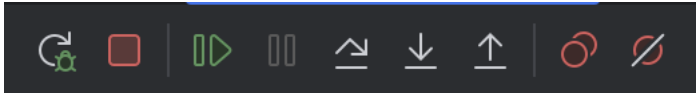
- Da wir sicher sind, dass **summe** und **anzahl** stimmen, wollen wir nun die Methode **findAverage** testen.

- Dazu setzen wir einen Breakpoint in der Zeile, in der diese Methode aufgerufen wird.


DebuggerExample
Version control
Current File
Project
DebuggerExample ~/IdeaP
.idea
out
src
DebuggerPart1
.gitignore
DebuggerExample.iml
External Libraries
Scratches and Consoles
DebuggerPart1.java
public class DebuggerPart1 {
public static void main(String[] args){
int zahl1 = 10;
int zahl2 = 20;
int zahl3 = 30;
int summe = zahl1+zahl2+zahl3;
int anzahl = 3;
int average = findAverage(summe, anzahl);
System.out.println("average:"+average);
}
public static int findAverage(int summe, int anzahl){ 1 usage
int result = summe * anzahl; //Es sollte summe/anzahl sein
return result;
}
}
Run
DebuggerPart1
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/lib/idea_rt.jar=53445 -Dfile.encoding=U
average:180
Process finished with exit code 0
DebuggerExample > src > DebuggerPart1
23:1 LF UTF-8 4 spaces


DebuggerExample
Version control
Current File
Project
DebuggerExample ~/IdeaP
.idea
out
src
DebuggerPart1
.gitignore
DebuggerExample.iml
External Libraries
Scratches and Consoles
DebuggerPart1.java
public class DebuggerPart1 {
public static void main(String[] args){ args: []
int zahl1 = 10; zahl1: 10
int zahl2 = 20; zahl2: 20
int zahl3 = 30; zahl3: 30
int summe = zahl1+zahl2+zahl3; summe: 60 zahl2: 20 zahl3: 30 zahl1: 10
int anzahl = 3; anzahl: 3
int average = findAverage(summe, anzahl); summe: 60 anzahl: 3
System.out.println("average:"+average);
}
public static int findAverage(int summe, int anzahl){ 1 usage
int result = summe * anzahl; //Es sollte summe/anzahl sein
return result;
}
}
Debug
DebuggerPart1
Threads & Variables
Console
"main"@1 i...n": RUNNING
Evaluate expression (⏎) or add a watch (⇧⌘⏎)
main:11, DebuggerPart1
args = {String[0]@823} []
zahl1 = 10
zahl2 = 20
zahl3 = 30
summe = 60
anzahl = 3
Switch frames from anywhere in the IDE wi...
DebuggerExample > src > DebuggerPart1
11:1 LF UTF-8 4 spaces

- **Blaue Zeile:** alles **bis und ohne** diese Zeile wurde ausgeführt
- **Debugger Symbole:**



View breakpoints
- Alle breakpoints anzeigen

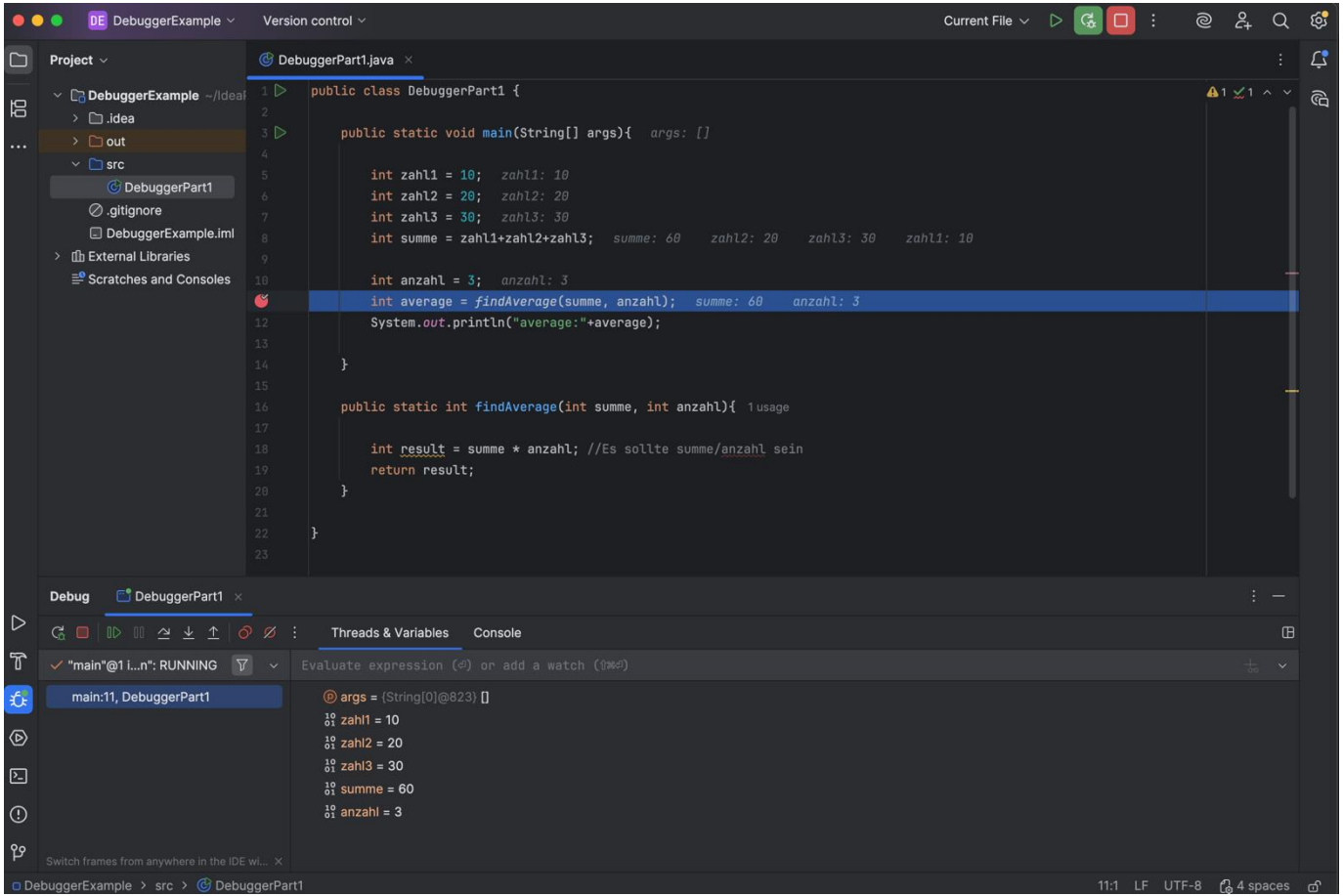
Step out
- Um aus eine Methode raus zu treten

Termination
- Um den Debugger zu stoppen

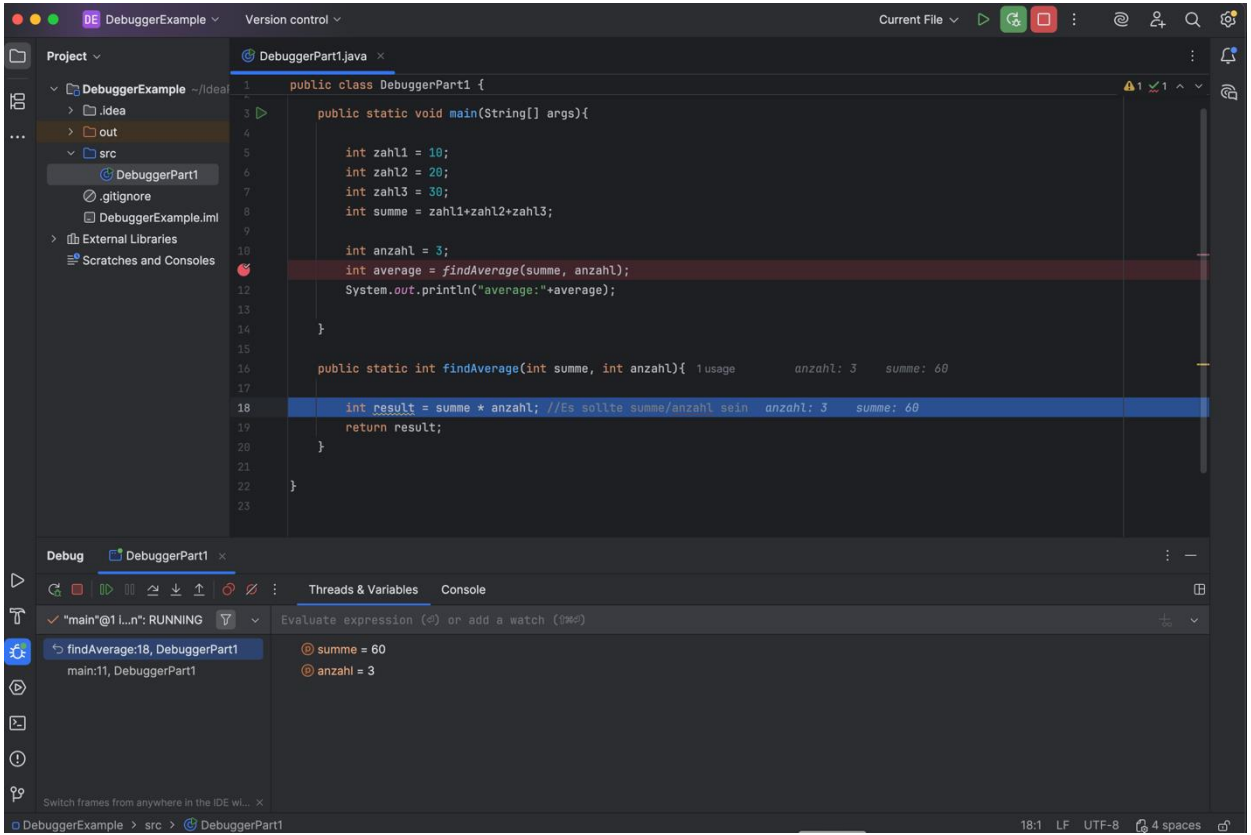
Step over
- Um zur nächsten Zeile zu gehen.

Step into
- Um in eine Methode zu springen.

Von dieser Zeile aus weitergehen…

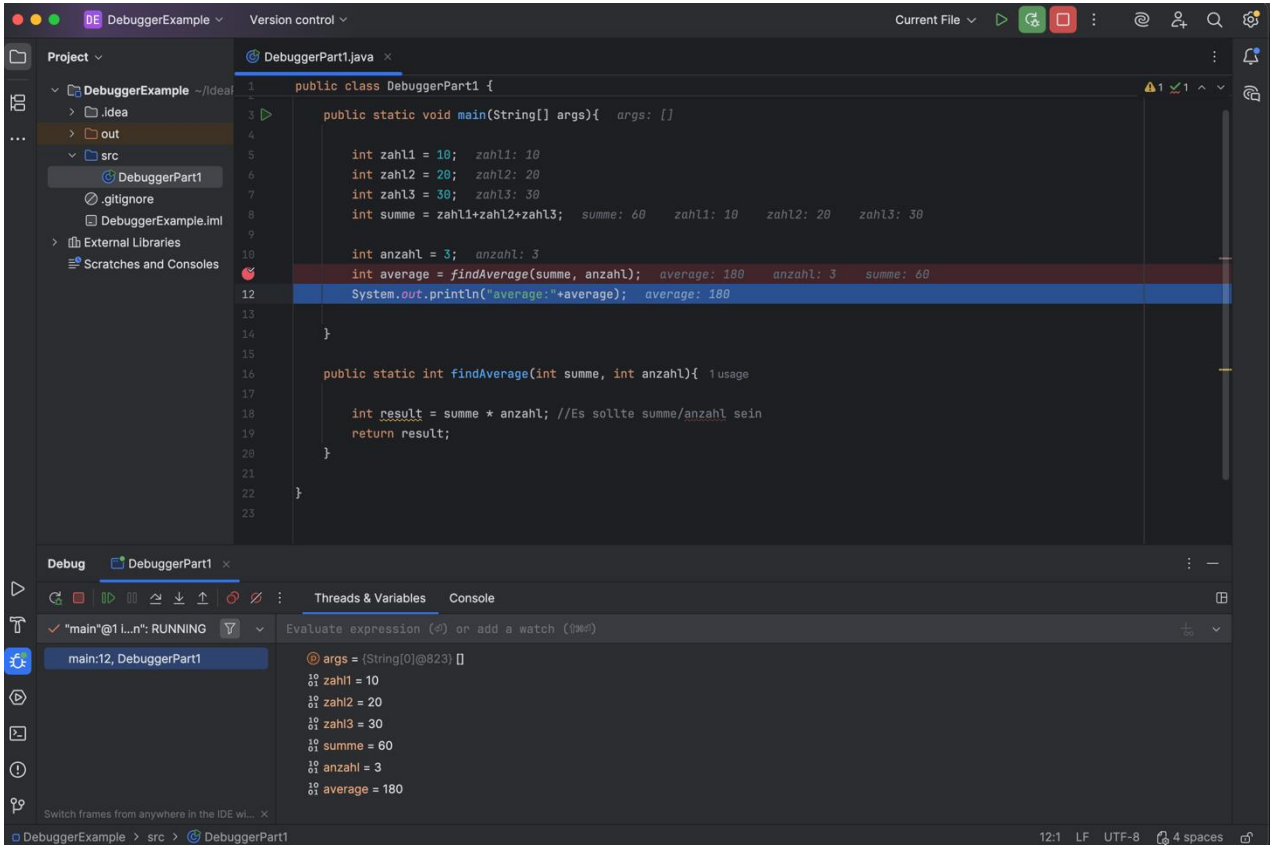


Mit step into:



Wir sind in die aufgerufene Methode **findAverage** hineingegangen und können nun diese Methode Schritt für Schritt ausführen.

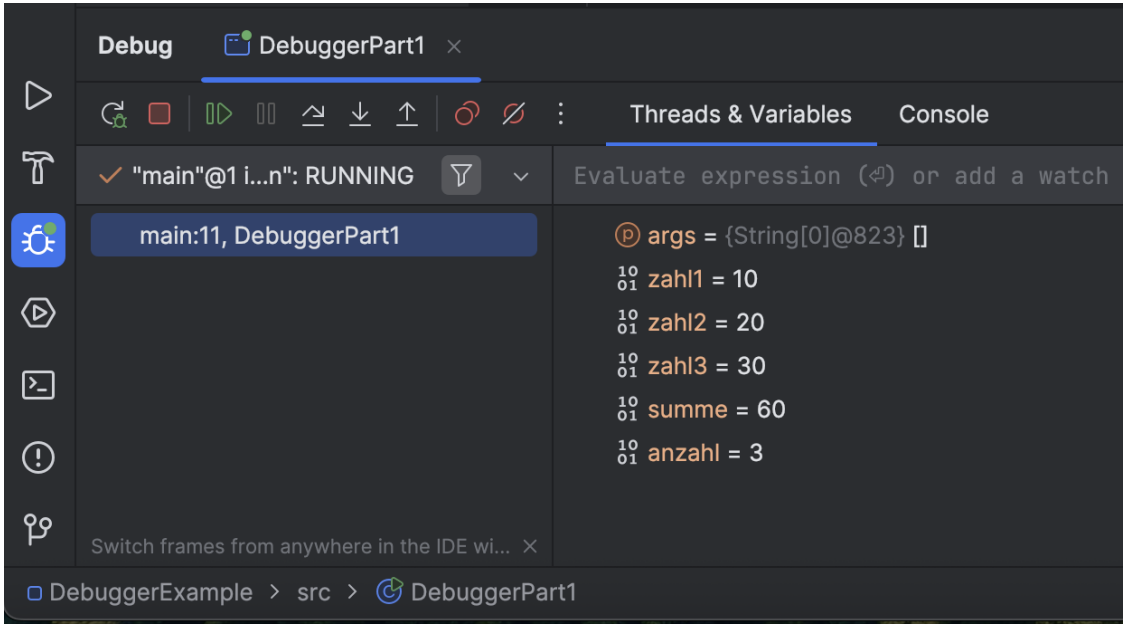
Mit step over:



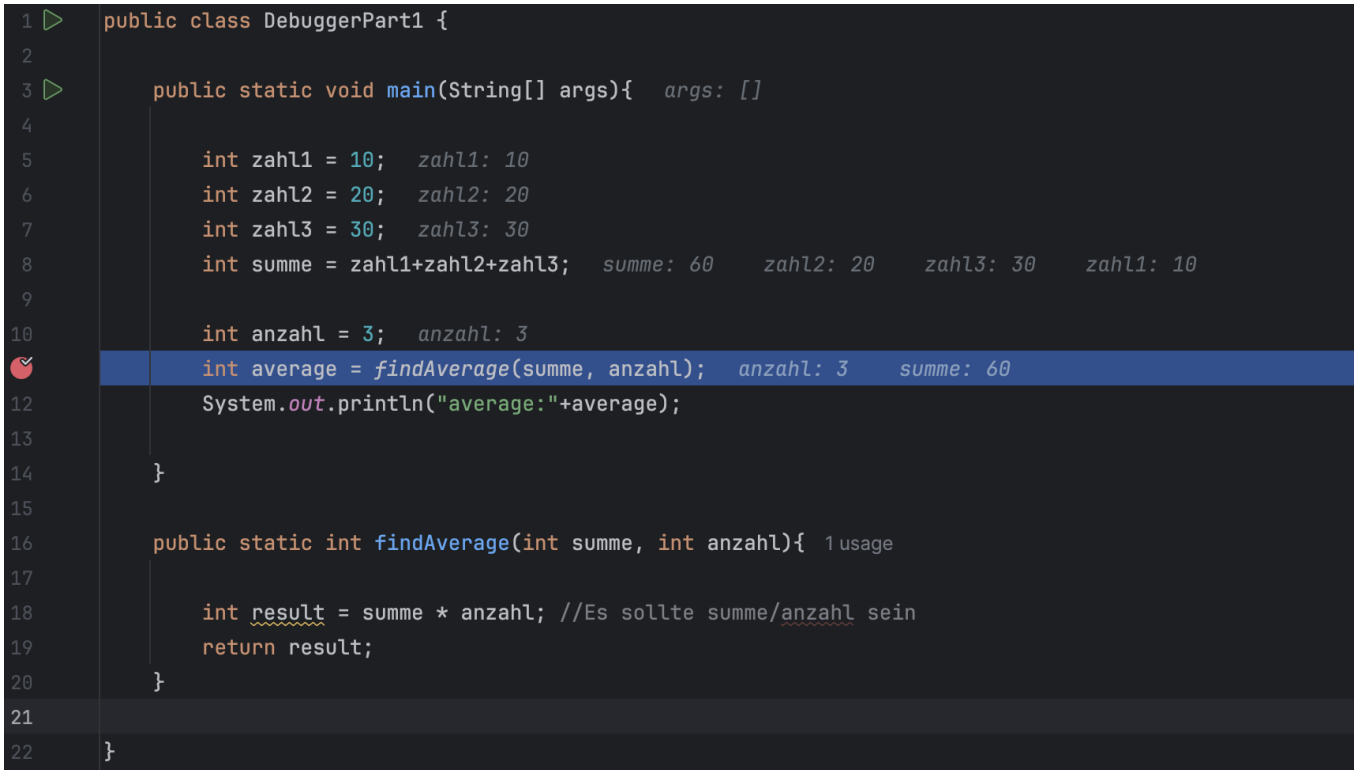
- Der Methodenaufruf von **findAverage** wurde in einem Schritt ausgeführt und der Rückgabewert an **average** zugewiesen.
- Wir befinden uns jetzt in der nächsten Zeile.

Step over vs. Step into: Wann welches benutzen?

- **Step over:** 
  - Zur nächsten Zeile springen.
  - Wenn in der Zeile eine Methode aufgerufen wird, wird sie ausgeführt. Man geht davon aus, dass sie wie erwartet funktioniert und dass der Rückgabewert korrekt ist.
- Step into: 
  - In die Methode hineingehen, um sie schrittweise zu durchlaufen.
  - Dies verwendet man, wenn man unsicher ist, ob die Methode wie erwartet funktioniert, und man den Ablauf genauer überprüfen möchte.



- Zeigt alle Variablen und ihre Werte an, die im Scope der blauen Zeile liegen.
- Nach der Ausführung jeder Zeile kann man überprüfen, ob sich die Werte wie erwartet geändert haben.



```java
public class DebuggerPart1 {

    public static void main(String[] args){   args: []

        int zahl1 = 10;   zahl1: 10
        int zahl2 = 20;   zahl2: 20
        int zahl3 = 30;   zahl3: 30
        int summe = zahl1+zahl2+zahl3;   summe: 60    zahl2: 20    zahl3: 30    zahl1: 10

        int anzahl = 3;   anzahl: 3
        int average = findAverage(summe, anzahl);   anzahl: 3    summe: 60
        System.out.println("average:"+average);

    }

    public static int findAverage(int summe, int anzahl){  1 usage

        int result = summe * anzahl; //Es sollte summe/anzahl sein
        return result;
    }

}
```

- Die Werte der Variablen werden auch am Ende jeder Zeile in Grau angezeigt

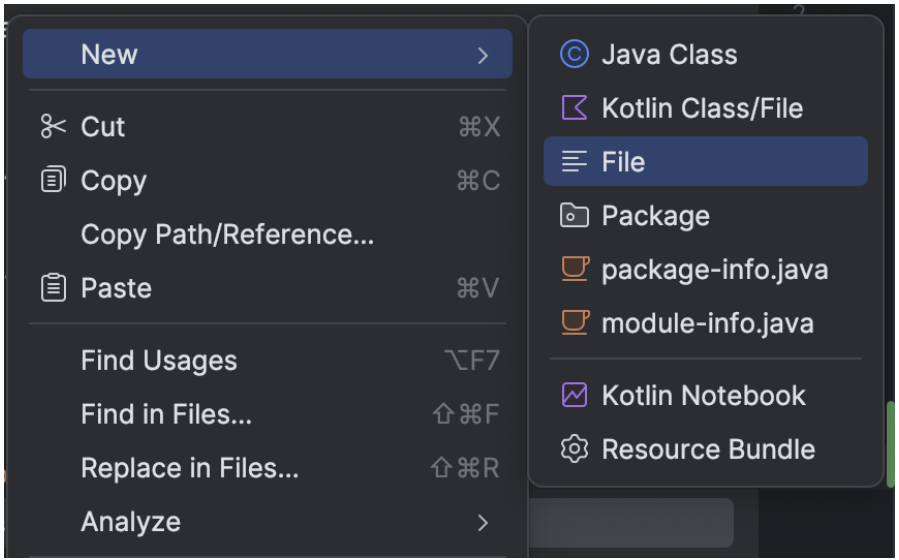
Feedback

Feedback

- Erstellt eine requestfeedback.txt Datei direkt in dem uXX Ordner, und schreibt hinein, zu welchen Aufgaben ihr Feedback haben wollt
- Bonus wird separat gegraded

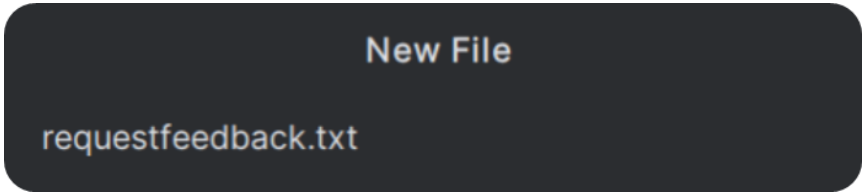
Feedback Datei erstellen

1. **Projekt Ordner uXX rechts-klicken.**
2. **New -> File.**



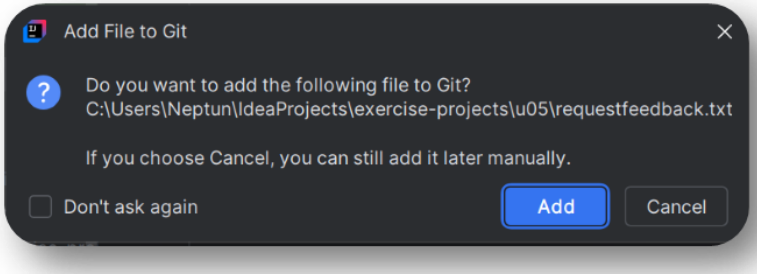
Feedback Datei erstellen

1. **Projekt Ordner uXX rechts-klicken.**
2. **New -> File.**
3. **Benenne die Datei als requestfeedback.txt im erscheinenden Fenster.**



Feedback Datei erstellen

1. Projekt Ordner uXX rechts-klicken.
2. New -> File.
3. Benenne die Datei als requestfeedback.txt im erscheinenden Fenster.
4. Wähle *Add* im auftauchenden Fenster «Add File to Git»



Nachbesprechung

Aufgabe 2: Grösster gemeinsamer Teiler

Schreiben Sie ein Programm "GGT.java", das den grössten gemeinsamen Teiler (ggT) zweier ganzer Zahlen mithilfe des Euklidischen Algorithmus berechnet. Hierbei handelt es sich um eine iterative Berechnung, die auf folgender Beobachtung basiert:

1. Wenn x grösser als y ist, dann ist — sofern sich x durch y teilen lässt — der ggT von x und y gleich y;
2. andernfalls ist der ggT von x und y der gleiche wie der ggT von y und $x \,\%\, y$.

```
while (x <= y || x % y != 0) { //solange x kleiner als y oder x sich nicht durch y teilen lässt:
    //GGT noch nicht gefunden
    //update x und y für nächste Iteration
    int altY = y; // Zwischenspeicher

    y = x % y;
    x = altY;
}

//GGT gefunden (x >= y && x % y == 0)
System.out.println(y);
```

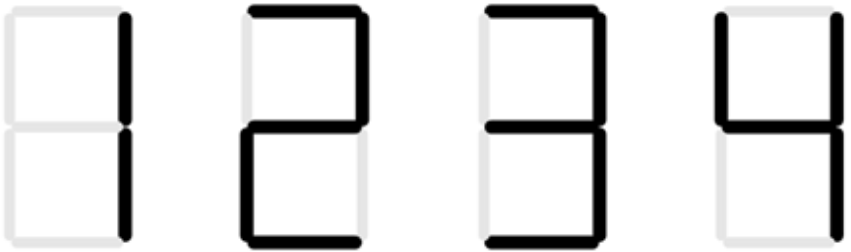
Negation der Bedingung in 1.

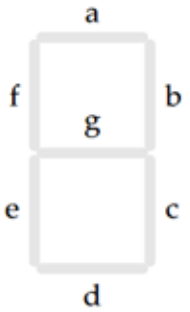
Aufgabe 3: Zahlenerkennung

Für diese Aufgabe verwenden wir einen String um die erleuchtenden Segmente einer Siebensegmentanzeige zu kodieren. Die Segmente sind, wie im Bild gezeigt, von a bis g nummeriert. Die Kodierung einer möglichen Anzeige ist ein String, in welchem der Buchstabe 'x' genau dann vorkommt, wenn das 'x'te Segment der Anzeige erleuchtet ist. Zum Beispiel wird die Zahl 2 kodiert durch 'abged'. Zur Einfachheit darf angenommen werden, dass kein Buchstabe mehr als einmal in der Kodierung vorkommt und dass nur die Zahlen 0 bis 9 kodiert werden.

Schreiben Sie ein Programm "Zahlen.java", das einen String, der eine Anzeige kodiert, einliest und die kodierte Zahl als Integer ausgibt. Überlegen Sie wie viele IF Blöcke benötigt werden um jede Zahl zu erkennen.

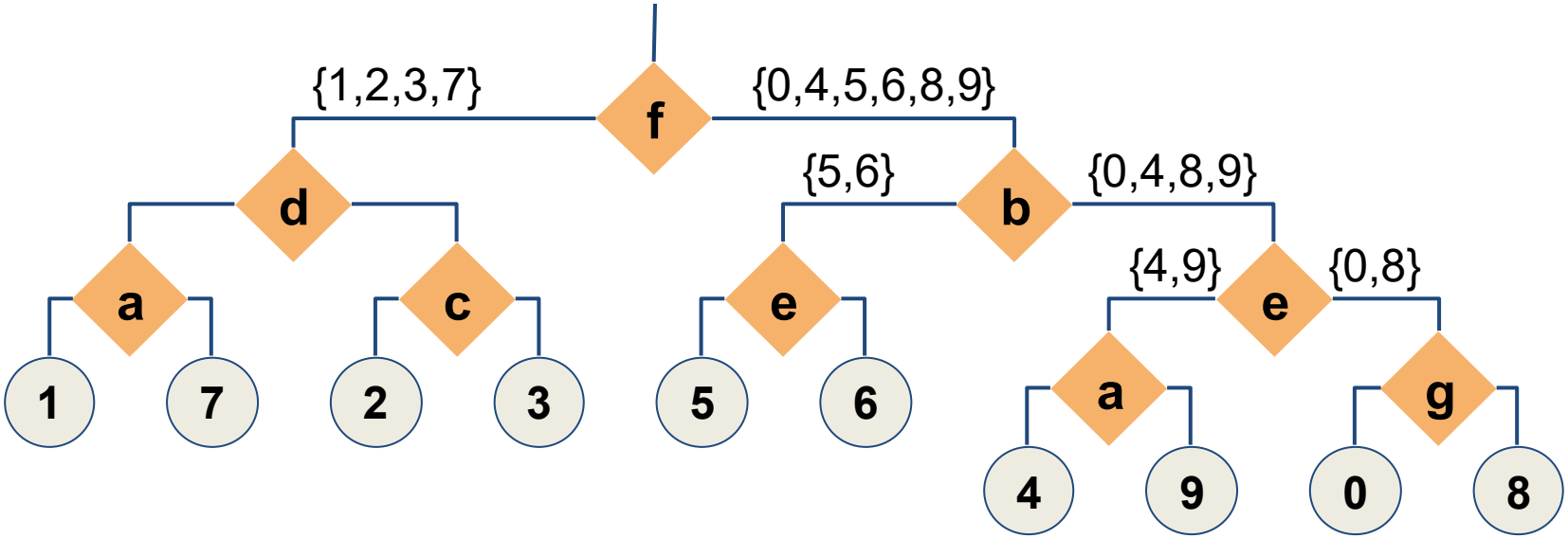
Tipp: Sie können `str.contains("a")` verwenden, um zu überprüfen, ob ein String `str` den Buchstaben 'a' enthält.





Aufgabe 3: Zahlenerkenn





Aufgabe 4: Berechnungen

2. Implementieren sie die Methode `Calculations.magic7(int a, int b)`. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn einer der Parameter 7 ist oder wenn die Summe oder Differenz der Parameter 7 ist. Ansonsten soll die Methode `false` zurückgeben.

   Beispiele

   - `magic7(2,5)` gibt `true` zurück.
   - `magic7(7,9)` gibt `true` zurück.
   - `magic7(5,6)` gibt `false` zurück.

   Hinweis: Mit der Funktion `Math.abs(num)` können Sie den absoluten Wert einer Zahl `num` erhalten.

Aufgabe 4: Berechnungen

3. Implementieren Sie die Methode `Calculations.fast12(int z)`. Das Argument `z` ist nicht negativ. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn `z` nahe an einem Vielfachen von `12` ist. Eine Zahl `x` ist nahe an einer Zahl `y`, wenn eine der Zahlen um maximal `2` grösser oder kleiner ist als die andere Zahl. Ansonsten soll die Method `false` zurückgeben.

   - `fast12(12)` gibt `true` zurück.
   - `fast12(14)` gibt `true` zurück.
   - `fast12(10)` gibt `true` zurück.
   - `fast12(15)` gibt `false` zurück.

2. Implementieren Sie die Methode `Calculations.magic7(int a, int b)`. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn einer der Parameter 7 ist oder wenn die Summe oder Differenz der Parameter 7 ist. Ansonsten soll die Methode `false` zurückgeben.

   Beispiele

   - `magic7(2,5)` gibt `true` zurück.
   - `magic7(7,9)` gibt `true` zurück.
   - `magic7(5,6)` gibt `false` zurück.

```java
public static boolean magic7(int a, int b) {  6 usages  zelleraa
    int sum = a + b;
    int diff1 = a - b;
    int diff2 = b - a;

    return a == 7 || b == 7 || sum == 7 || diff1 == 7 || diff2 == 7;
}
```

   Hinweis: Mit der Funktion `Math.abs(num)` können Sie den absoluten Wert einer Zahl `num` erhalten.

3. Implementieren Sie die Methode `Calculations.fast12(int z)`. Das Argument z ist nicht negativ. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn z nahe einem Vielfachen von 12 ist. Eine Zahl x ist nahe an einer Zahl y, wenn eine der Zahlen um maximal 2 grösser oder kleiner ist als die andere Zahl. Ansonsten soll die Methode `false` zurückgeben.

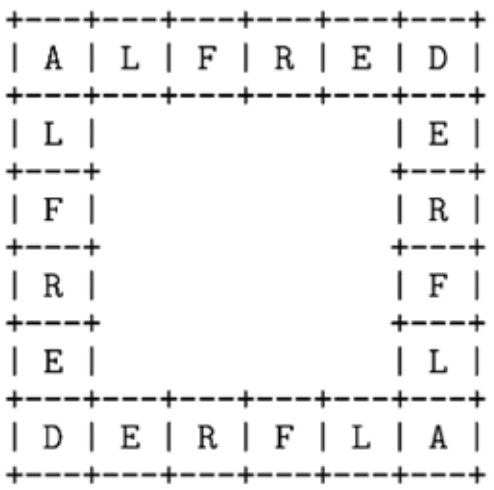
   Beispiele

   - `fast12(12)` gibt `true` zurück.
   - `fast12(14)` gibt `true` zurück.
   - `fast12(10)` gibt `true` zurück.
   - `fast12(15)` gibt `false` zurück.

```java
public static boolean fast12(int z) {  1 usage
    return Math.abs(z % 12) <= 2 || Math.abs(z % 12) >= 10;
}
```

Aufgabe 5: Scrabble

In dieser Aufgabe sollen Sie Scrabble-Steine legen, mittels ASCII-Art auf der Konsole. Vervollständigen Sie die Methode `drawNameSquare` in der Klasse `Scrabble`. Diese Methode nimmt einen Namen als String-Parameter und soll den Namen als in einem Quadrat angeordnete Scrabble-Steine auf der Konsole (`System.out`) ausgeben. Wenn z.B. der String `Alfred` übergeben wird, sollte folgendes Bild ausgegeben werden:



Vorbesprechung

Aufgabe 1: Sieb des Eratosthenes

Schreiben Sie ein Programm "Sieb.java", das eine Zahl *limit* einliest und die Anzahl der Primzahlen, die grösser als 1 und kleiner oder gleich dem *limit* sind, ausgibt. Dazu ermitteln Sie in einem ersten Schritt alle Primzahlen, die kleiner oder gleich *limit* sind. Dieses Teilproblem können Sie mit dem Sieb des Eratosthenes lösen. Das Sieb des Eratosthenes findet Primzahlen bis n. Man betrachtet alle Zahlen von 2 bis `n` und streicht zuerst alle Vielfachen der ersten Zahl (2). Dann geht man zur nächsten ungestrichenen Zahl (3) und wiederholt das Streichen ihrer Vielfachen. Das macht man, bis man dies für alle Zahlen gemacht hat. Sie können ein `Boolean`-Array verwenden, um zu speichern, welche Zahlen Primzahlen sind und welche nicht. Übrig bleiben die Primzahlen. Danach können Sie die Anzahl der gefundenen Primzahlen anhand dieses Arrays bestimmen.

Beispiel: Für $limit = 13$ sollte Ihr Programm 6 ausgeben (Primzahlen: 2, 3, 5, 7, 11, 13).

Hinweis: Es ist nicht zwingend nötig von 2 bis n zu gehen. Von 2 bis $\sqrt{n}$ zu gehen reicht bereits aus, da eine Zahl $\leq n$ nicht einen Teiler grösser als $\sqrt{n}$ ausser sich selbst haben kann.

Aufgabe 2: Arrays



1. Implementieren Sie die Methode `ArrayUtil.zeroInsert(int[] x)` in der Datei "ArrayUtil.java". Die Methode nimmt einen Array `x` als Argument und gibt einen Array zurück. Der zurückgegebene Array soll die gleichen Werte wie `x` haben, ausser: Wenn eine positive Zahl direkt auf eine negative Zahl folgt oder wenn eine negative Zahl direkt auf eine positive Zahl folgt, dann wird dazwischen eine 0 eingefügt.

   Beispiele:

   - Wenn `x` gleich `[3, 4, 5]` ist, dann wird `[3, 4, 5]` zurückgegeben.
   - Wenn `x` gleich `[3, 0, -5]` ist, dann wird `[3, 0, -5]` zurückgegeben.
   - Wenn `x` gleich `[-3, 4, 6, 9, -8]` ist, dann wird `[-3, 0, 4, 6, 9, 0, -8]` zurückgegeben.

   Versuchen Sie, die Methode rekursiv zu implementieren.

Aufgabe 2: Arrays



2. Implementieren Sie die Methode `ArrayUtil.tenFollows(int[] x, int index)`. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn im Array `x` ab Index `index` der zehnfache Wert einer Zahl `n` direkt der Zahl `n` folgt. Dies muss nur für das erste Auftreten der Zahl `n` ab Index `index` im Array `x` geprüft werden. Ansonsten soll die Methode `false` zurückgeben.

   Beispiele:

   - `tenFollows([1, 2, 20], 0)` gibt `true` zurück.
   - `tenFollows([1, 2, 7, 20], 0)` gibt `false` zurück.
   - `tenFollows([3, 30], 0)` gibt `true` zurück.
   - `tenFollows([3], 0)` gibt `false` zurück.
   - `tenFollows([1, 2, 20, 5], 1)` gibt `true` zurück.
   - `tenFollows([1, 2, 20, 5], 2)` gibt `false` zurück.

   Die `main` Methode in `ArrayUtil` gibt die oben genannten Beispielaufrufe sowie das entsprechende Ergebnis der jeweiligen Methode aus. Hiermit können Sie überprüfen, ob Ihre Implementierungen die richtigen Ergebnisse zurückliefern. In "ArrayUtilTest.java" im Ordner "test" in der Übungsvorlage finden Sie zusätzlich einige Unit-Tests für beide Methoden (für eine detaillierte Beschreibung zu automatisiertem Testen und der Ausführung solcher Tests siehe Aufgabe 3). Sie können die `main` Methode und die Tests beliebig abändern und/oder mit Ihren eigenen Inputs erweitern.

Aufgabe 3: 2D Arrays



Gegeben einer Matrix M, prüfen Sie zuerst ob diese eine $n \times n$ Matrix ist, deren Elemente positive ganze Zahlen sind. Danach prüfen Sie ob zusätzlich alle Zahlen kleiner gleich n^2 sind. Somit gilt nun $0 < m_{i,j} \leq n^2$. Prüfen Sie ebenfalls, ob die Elemente der Matrix jeweils genau einmal vorkommen, sprich ob $m_{x,y} = m_{p,q} \Rightarrow (x = p) \wedge (y = q)$ gilt. Wir sagen, dass die Matrix M *perfekt* ist, wenn zusätzlich alle Zeilensummen und Spaltensummen gleich sind (also $\sum_{k=0}^{k=n-1} m_{i,k} = \sum_{k=0}^{k=n-1} m_{j,k}$ für alle i, j und $\sum_{k=0}^{k=n-1} m_{k,i} = \sum_{k=0}^{k=n-1} m_{k,j}$ für alle i, j mit $0 \leq i, j < n$).

Vervollständigen Sie die Methode `boolean checkMatrix(int[][] m)` von der Klasse Matrix, so dass diese Methode `true` zurückgibt wenn die Input Matrix *perfekt* ist, und `false` sonst. Sie können davon ausgehen, dass der Parameter `m` nicht `null` ist. Alle anderen Eigenschaften müssen Sie selber testen. Eine Matrix ist nur perfekt, wenn alle genannten Eigentschaften gelten.

Testen Sie Ihr Programm ausgiebig - am besten mit JUnit - und pushen Sie die Lösung vor dem Abgabetermin. Wir haben Ihnen einen JUnit Test in der Klases `MatrixTest` bereits erstellt.

Aufgabe 4: Testen mit JUnit

Zweck des Programms:

- Wochentag eines Datums (nach 01.01.1900) ausgeben
  Beispiel:13.10.2017 → Friday
  Gibt fälschlicherweise aber *“The 13.10.2017 is a Sunday”* aus.
- Berücksichtigt Schaltjahre (“Leap year”)

Funktionsweise:

1. Überprüft, ob Datum OK ist
2. Zählt die Tage ab 1.1.1900 bis zum eingegebenen Datum
3. Wochentag = Tage % 7

Aufgabe 5: Matching Numbers ★★★★☆

Implementieren Sie die Methode `Match.matchNumber(long A, int M)`. Die Methode soll für eine Zahl `A` und eine nicht-negative drei-stellige Zahl `M` die Position von `M` in `A` zurückgeben. Sei `M` eine Zahl mit den Ziffern $M_2M_1M_0$ (das heisst, es gilt $M = M_0 + 10 \cdot M_1 + 100 \cdot M_2$), wobei jede Ziffer 0 sein kann. Zusätzlich sei `A` eine Zahl, sodass A_i die i-te Ziffer von `A` ist (das heisst, es gilt $|A| = \sum_i 10^i \cdot A_i$), wobei `A` unendlich viele führende Nullen hat. Die Position von `M` in `A` ist die kleinste Zahl j, sodass $A_j = M_0$ und $A_{j+1} = M_1$ und $A_{j+2} = M_2$ gilt. Die Methode soll `-1` zurückgeben, falls es kein solches j gibt.

Beispiele:

`matchNumber(32857890, 789)` soll `1` zurückgeben.
`matchNumber(37897890, 789)` soll `1` zurückgeben.
`matchNumber(1800765, 7)` soll `2` zurückgeben.
`matchNumber(1800765, 8)` soll `-1` zurückgeben (die drei Ziffern von `8` sind `008`).
`matchNumber(75, 7)` soll `1` zurückgeben (da `007` and Position `1` von `0075` ist).

Aufgabe 5: Matching Numbers ★★★★☆

Implementieren Sie die Berechnung in der Methode `int matchNumber(long A, int M)`, welche sich in der Klasse `Match` befindet. Die Deklaration der Methode ist bereits vorgegeben. Sie können davon ausgehen, dass $0 \leq \texttt{M} < 1000$ gilt.

In der `main` Methode der Klasse `Match` finden Sie die oberen Beispiele als kleine Tests, welche Beispiel-Aufrufe zur `matchNumber`-Methode machen und welche Sie als Grundlage für weitere Tests verwenden können. In der Datei `MatchTest.java` geben wir die gleichen Tests zusätzlich auch als JUnit Test zur Verfügung. Sie können diese ebenfalls nach belieben ändern. Es wird *nicht* erwartet, dass Sie für diese Aufgabe den JUnit Test verwenden.

Tipp: Die Methode `Integer.toString(int i)` wandelt einen Integer in einen String um.

BONUS: Substring-Counter

- Diese Aufgabe gibt Bonuspunkte
- Plant Reserve-Zeit für technische Probleme ein
- Wenn ihr euch unsicher seid, ob der Commit & Push funktioniert hat, schaut auf GitLab online nach
- Genau die Aufgabenstellung lesen
- Regeln durchgehen: https://lec.inf.ethz.ch/infk/eprog/2026/exercises/additionals/bonusaufgaben_regeln.pdf

Danke!